GPIO: General Purpose Input/Output

ECE 2564, Embedded Systems

Fall 2025

GPIO: Introduction

GPIO: General Purpose Input/Output

- GPIO is the simplest and possibly the most versatile communication method between the microprocessor and the outside world
- We must write C-language code to *write* digital values and *read* digital values from various *memory mapped* registers
- These registers contain bits that correspond to physical devices such as pushbutton switches and LEDs, devices which are with ON or OFF

Coming Attractions ...

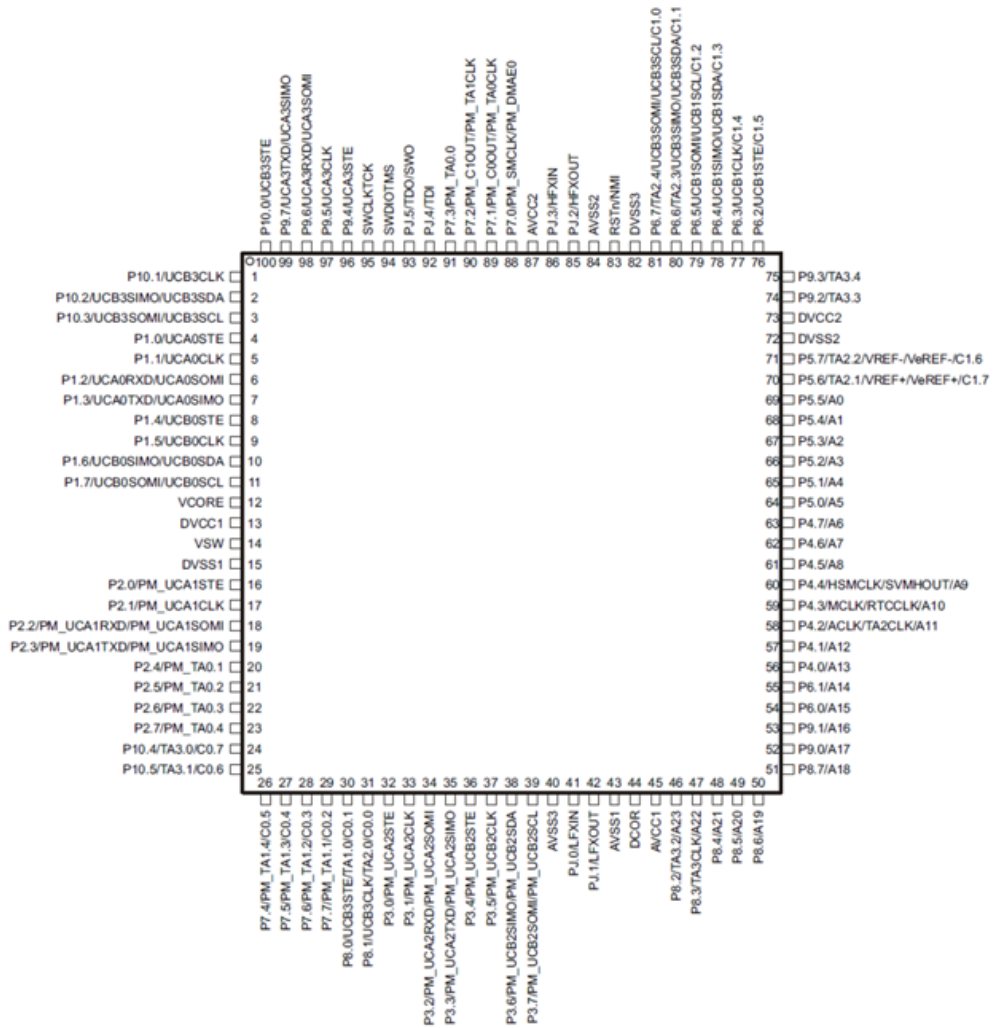
- Read the state of a switch:
  - `switch_state = *P1IN_addr & S1_MASK;`
- Turn on an LED:
  - `*P1OUT_addr = *P1OUT_addr | LED1_MASK;`
  - `*P1OUT_addr |= LED1_MASK; // alternative code`

Motivation

- Our microcontroller IC package has *physical pins*
- Our microcontroller's GPIO subsystem has logically defined "ports" and "pins"
- Pin: 1 bit
- Port: a collection of up to 8 pins
- Pin configuration
  - Some pins are only for input
  - Some pins are only for output
  - Some pins are bidirectional and are configured by software to be either input or output
- Naming convention: Px.y refers to Port x, Pin y
- **MCU Datasheet, Chapter 5**
- **MCU Technical Reference Manual, Chapter 12**

MSP-EXP432 Pinout[1]

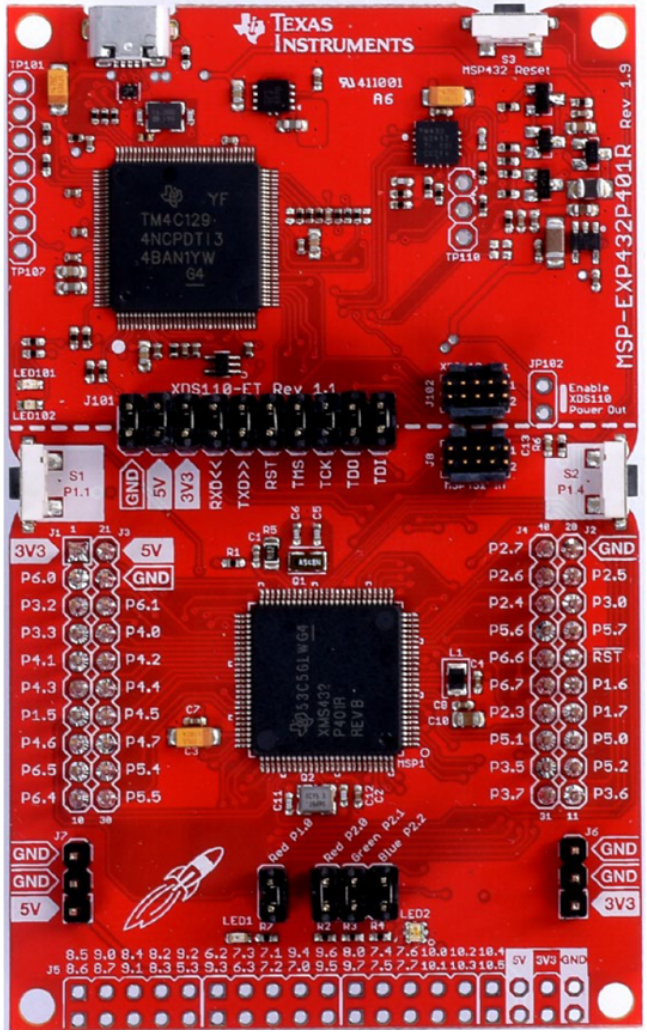
For each physical pin shown in this diagram the name of the primary function is shown first followed by the names of its other functions



[1]Source: LaunchPad User Guide

MSP-EXP432P401R LaunchPad Development Kit[2]

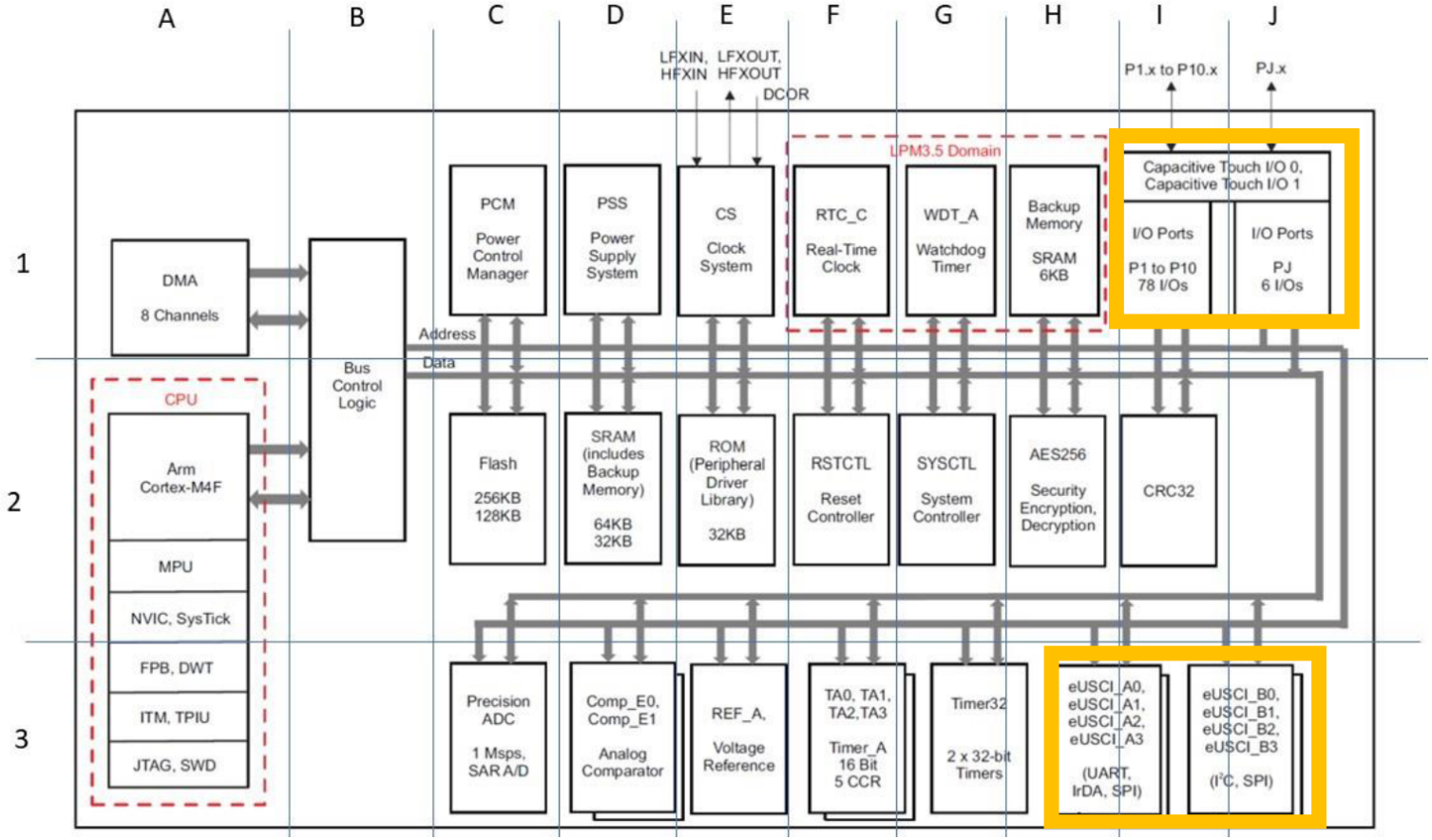
Let's find the switches, jumpers, and LEDs on the LaunchPad



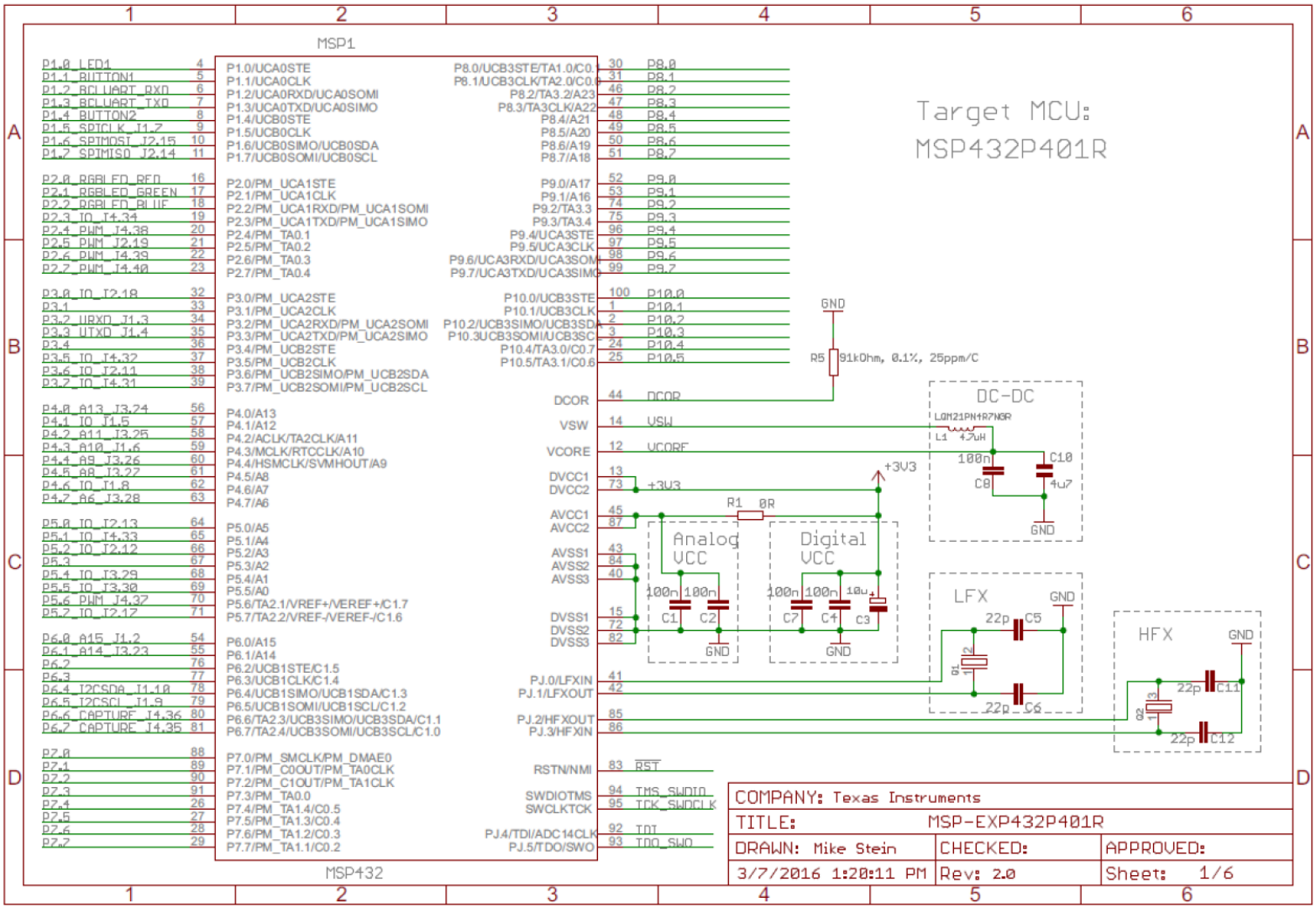
[2]Source: LaunchPad User Guide

MSP432 Functional Diagram

MSP432 – Digital Communication

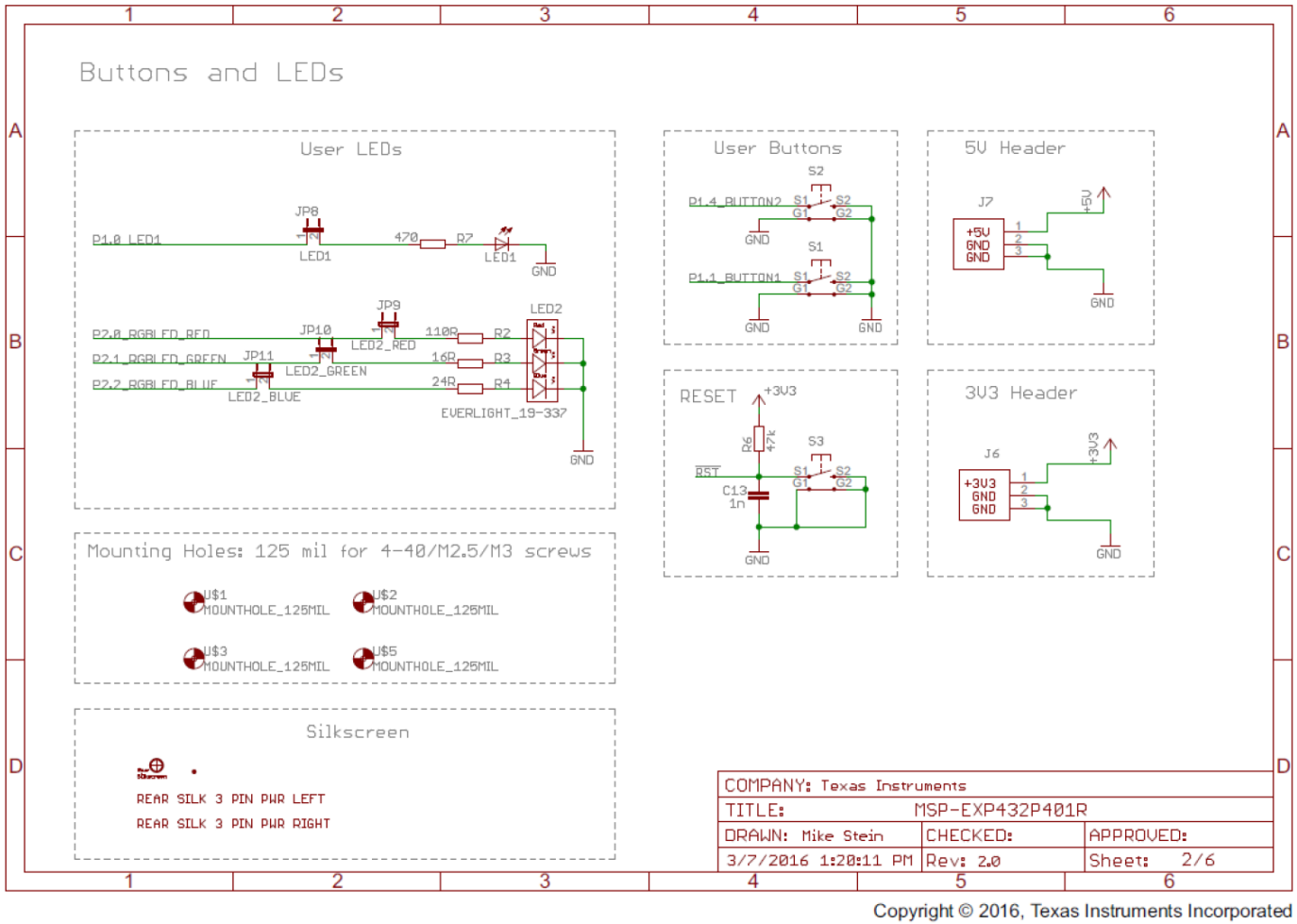


MSP432 Schematics Example



Copyright © 2016, Texas Instruments Incorporated

MSP432 Buttons and LEDs Schematics

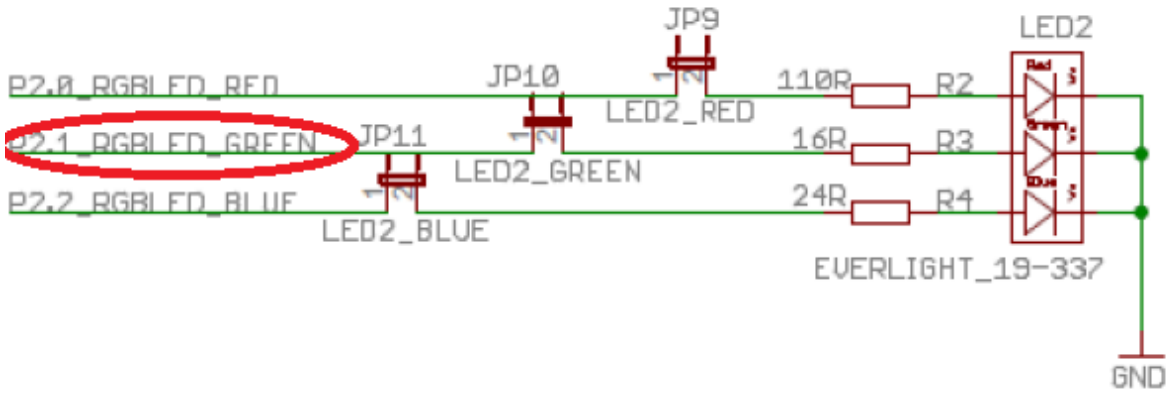


Copyright © 2016, Texas Instruments Incorporated

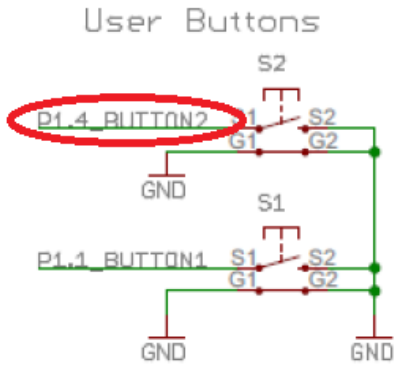
Schematic Diagrams for MSP432P40R Buttons and LEDs

Let's Identify some Port/Pin Pairs

- Which port/pin is the green LED (an output)?



- Which port/pin is Button #2 (an input)?

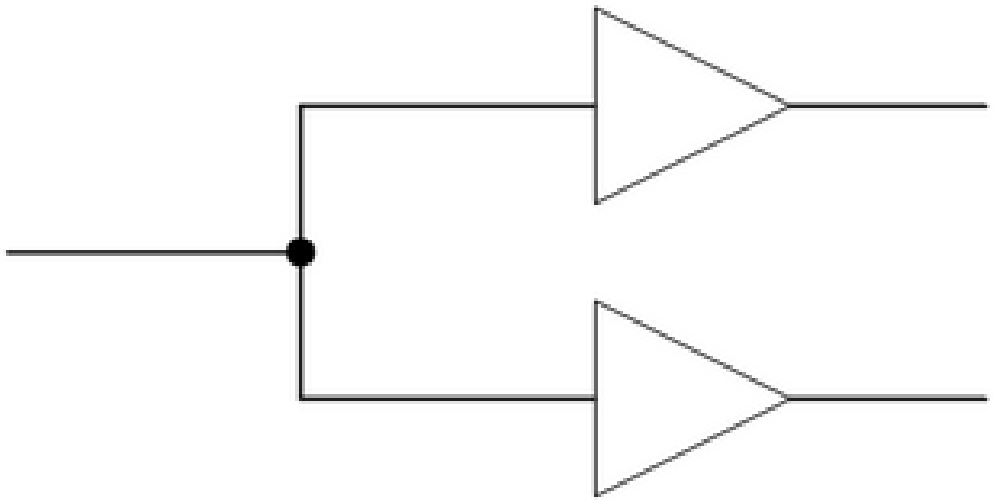


Configuring GPIOs

- When a physical pin is acting as a GPIO "pin" it is in direct communication with the CPU
- When a physical pin is acting in some other role it communicates with a *peripheral* device such as an ADC, timer, UART/I2C, etc.
- At a given time a pin can act as an input or an output
- Why cannot a pin be an input and an output at the same time?

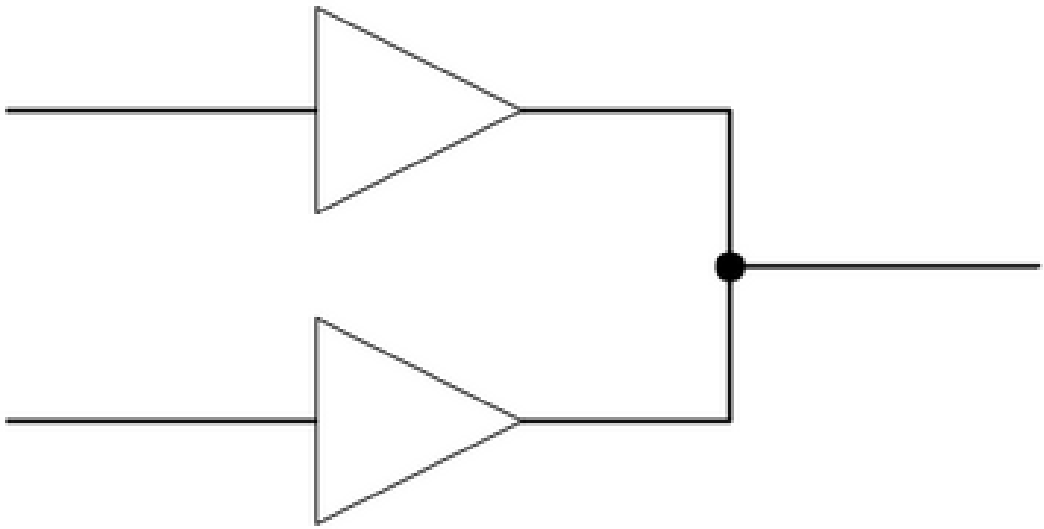
Configuring GPIOs

Is it OK to connect inputs of two gates?



Configuring GPIOs

Is it OK to connect outputs of two gates?



Configuring GPIOs: Designer's Challenge

- A pin cannot be an input and an output at the same time
- Option 1: Dedicate some pins as input and some pins as output
  - This reduces the flexibility of the design
- Option 2 (better): Design pins that can be configured as an input or as an output based on the programmer's needs
- Option 3 (even better): Option 2 + the ability for pins to also act as and analog input or carry out functions other than simple digital input or output

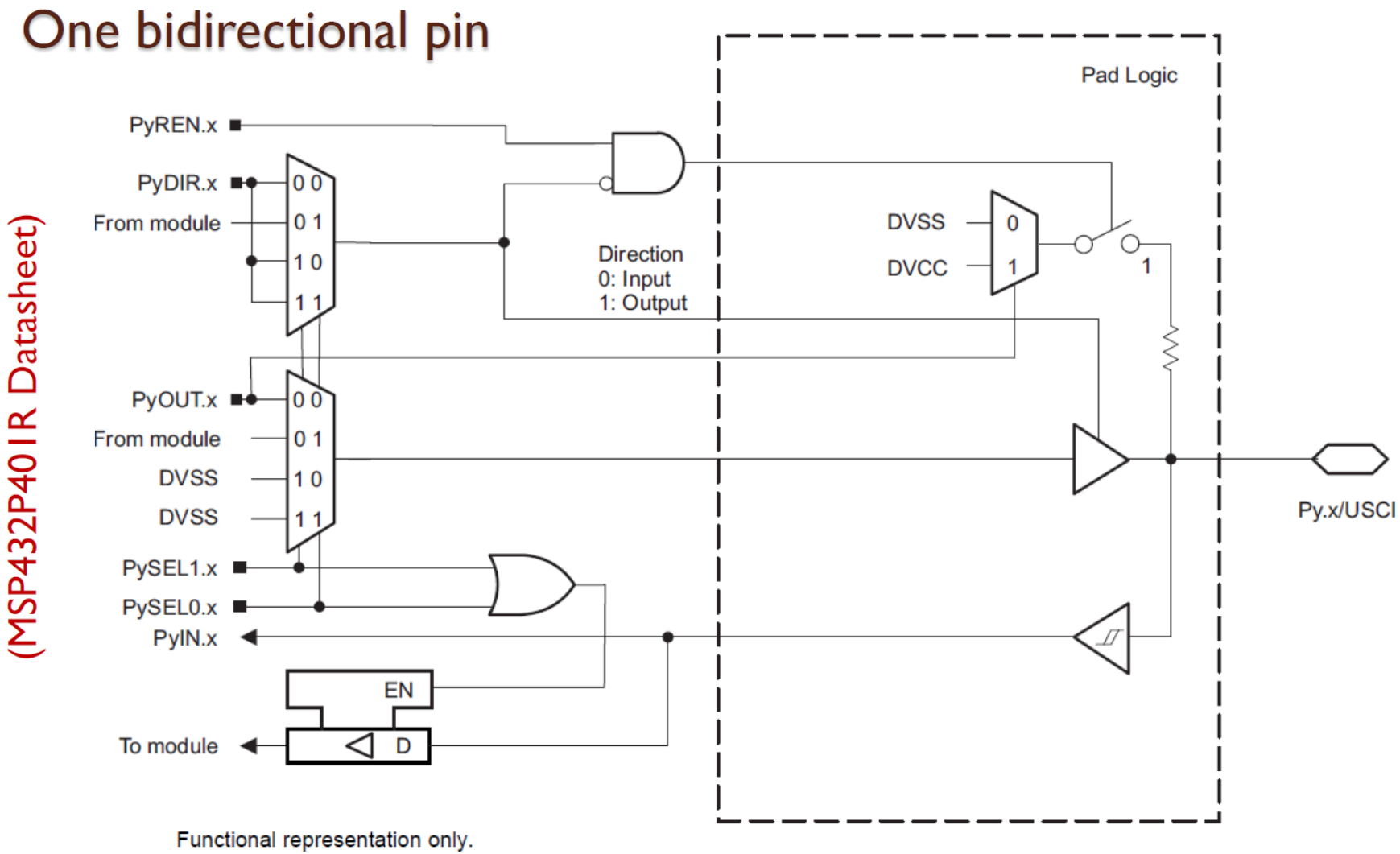
Configuring GPIOs

- How do pins on your MSP432 act as input sometimes and output sometimes?
- How do pins on your MSP432 act like digital I/O at times and take on other functionalities, such as analog input, at other times?
  The pins on your MSP432 are not just a simple piece of metal. Let's explore!

Notation:

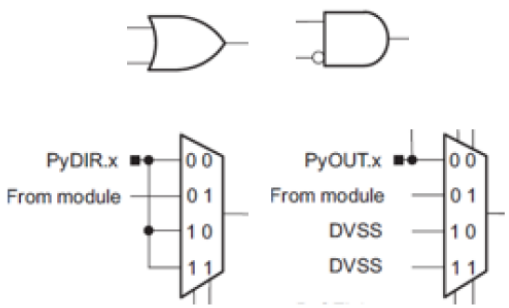
- **DIR**: DIRection
- **IN**: INput
- **OUT**: OUTput
- **REN**: Resistor ENable
- **SEL**: SELect

What's Inside a Bidirectional Pin

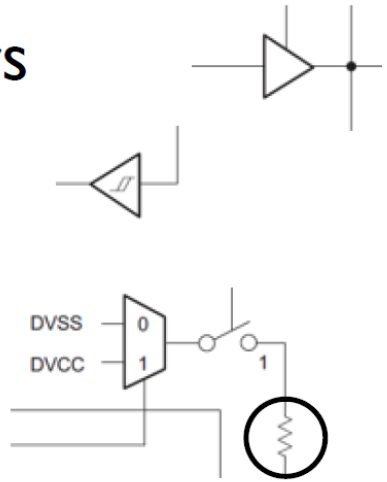


Lots of Components

- Basic logic gates
- Multiplexers



- Three-state buffers
- Schmitt triggers
- Pull-up or pull-down resistor

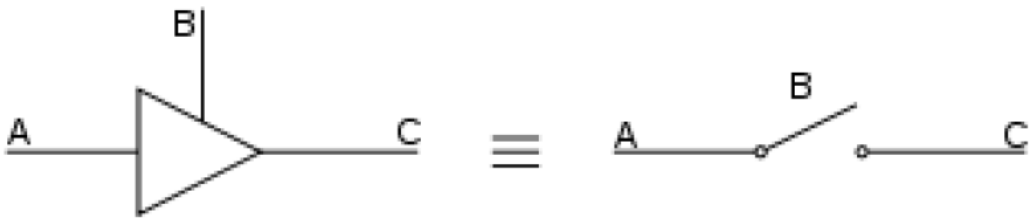


Port/Reg/Pin notation:

- PyZZZ.x
  - Port y, pin x, reg ZZZ
- P1Dir.2
  - Port 1, pin 2, reg Dir

- Configuration registers (not shown here)

Tri-State Buffers



INPUT		OUTPUT
A	B	C
0	0	Z (high impedance)
1		Z (high impedance)
0	1	0
1		1

I/O Port Registers

- **SEL1**, **SEL0** (SELect)
  - When SEL1 and SEL0 are both 0 the pin acts as digital I/O
  - Otherwise the pin acts in one of its other roles
- **DIR** (DIRection)
  - When DIR is 0 the corresponding pin is an input
  - When DIR is 1 the corresponding pin is an output
- **REN** (Resistor ENable)
  - This is relevant only when the corresponding pin is an input
  - When REN is 0 no pull-up/pull-down resistor is enabled
  - When REN is 1 a pull-up/pull-down resistor is enabled

I/O Port Registers

OUT (OUTput)

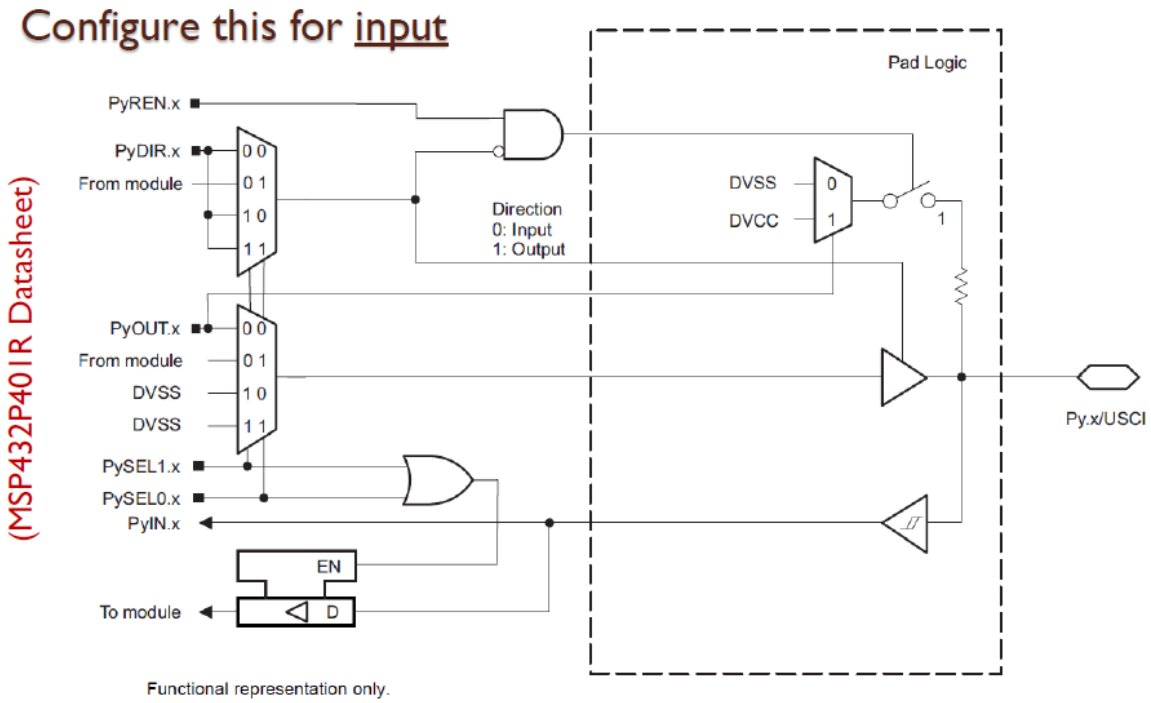
- *If the I/O pin is configured for output*, OUT holds the value that we want to send from the CPU to the physical pin, i.e., the value that is going to the outside world
- To light and LED we write code that sets values in the OUT register bits
- HOWEVER!!!
  - *If the I/O pin is configured as an input and a pull-up/pull-down resistor is enabled*, OUT controls the type of pull resistor
    - When OUT is 0 a pull-down resistor is enabled
    - When OUT is 1 a pull-up resistor is enabled

I/O Port Registers

IN (INput)

- *If the I/O is configured as an input*, IN holds the input value from the pin, the value that is coming from the outside world, for the CPU
- To know what a pushbutton's state is, we write code that reads values from the IN register bits
- *If the I/O is configured as an output*, IN does nothing

Configuring a Pin for Input



SEL0 = 0; SEL1 = 0

(MSP432P401R User Guide)

PxDIR	PxREN	PxOUT	I/O Configuration
0	0	x	Input
0	1	0	Input with pulldown resistor
0	1	1	Input with pullup resistor
1	x	x	Output

Using I/O Port Registers

- PyDIR, PyREN and PyOUT are write-only registers
- PyIN is a read-only register

Steps for using GPIO

The process for using GPIO for digital I/O or for using a peripheral involve the same steps:

- Configuring the behavior of a certain pin or peripheral
- Actually using the pin or peripheral for a useful purpose by reading ro writing to the register that corresponds to the pin or peripheral

Putting it All Together: Configuring a GPIO Pin for Port y, Pin x

- PySEL1.x, PySel1.x should be cleared to 0
- PyDIR.x:
  - 0 – Py.x is an input
  - 1 – Py.x is an output
- PyREN.x, *if Py.x is an input:*
  - 0 – The input pin does not use a pull-up/pull-down
  - 1 – The input pin use a pull-up/pull-down
- PyOUT.x, *if Py.x is an input and PyREN.x = 1*
- 0 – The input uses a pull-down resistor
- 1 – The input uses a pull-up resistor

If Py.x is an output, the values of PyRES.x and PyOUT.x are irrelevant with respect to configuration

Putting it All Together: Using a GPIO Pin for Port y, Pin x

- PyIN.x, *if Py.x is an input:*
  - Read the value in PyIN.x to determine the state of the input device interfaced with Py.x
  - Interpret the input, 0 or 1, appropriately for the device interfaced with Py.x
- PyOUT.x, *if Py.x is an output:*
  - Write a value to PyOUT.x to set the desired state of the output device interfaced to pin Py.x
  - For outputs like LEDs, usually a 0 will turn the LED off and a 1 will turn the LED on

I/O Configuration

I/O Configuration

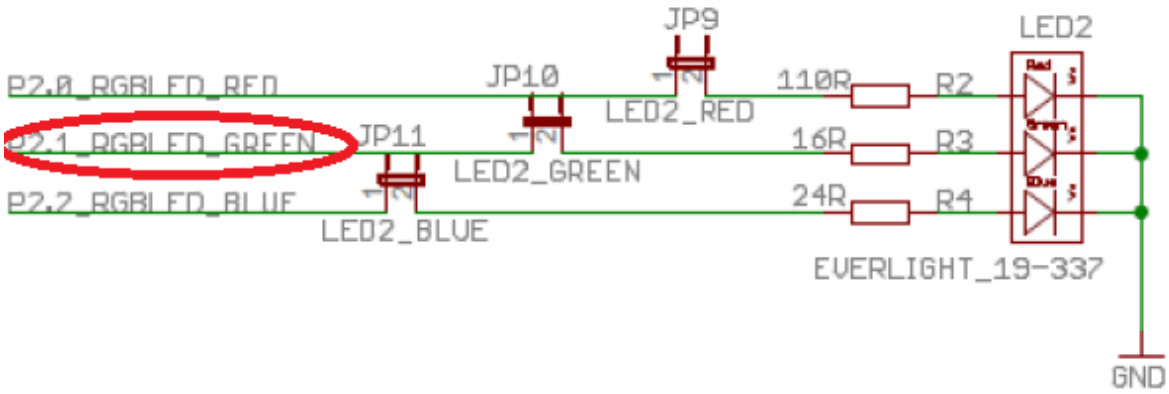
PxDIR	PxREN	PxOUT	I/O Configuration
0	0	x	Input
0	1	0	Input with pulldown resistor
0	1	1	Input with pullup resistor
1	x	x	Output

I/O Configuration

I/O Configuration

Register	Primary Function	Secondary Function
PxIN	Incoming value	
PxOUT	Outgoing Value	If xDIR == 0 && xREN ==1 indicates PU or PD
PxDIR	0 = input, 1 = output	
PxREN	0 = No PU/PD, 1 = PU/PD	
PxSEL	00 = Port	

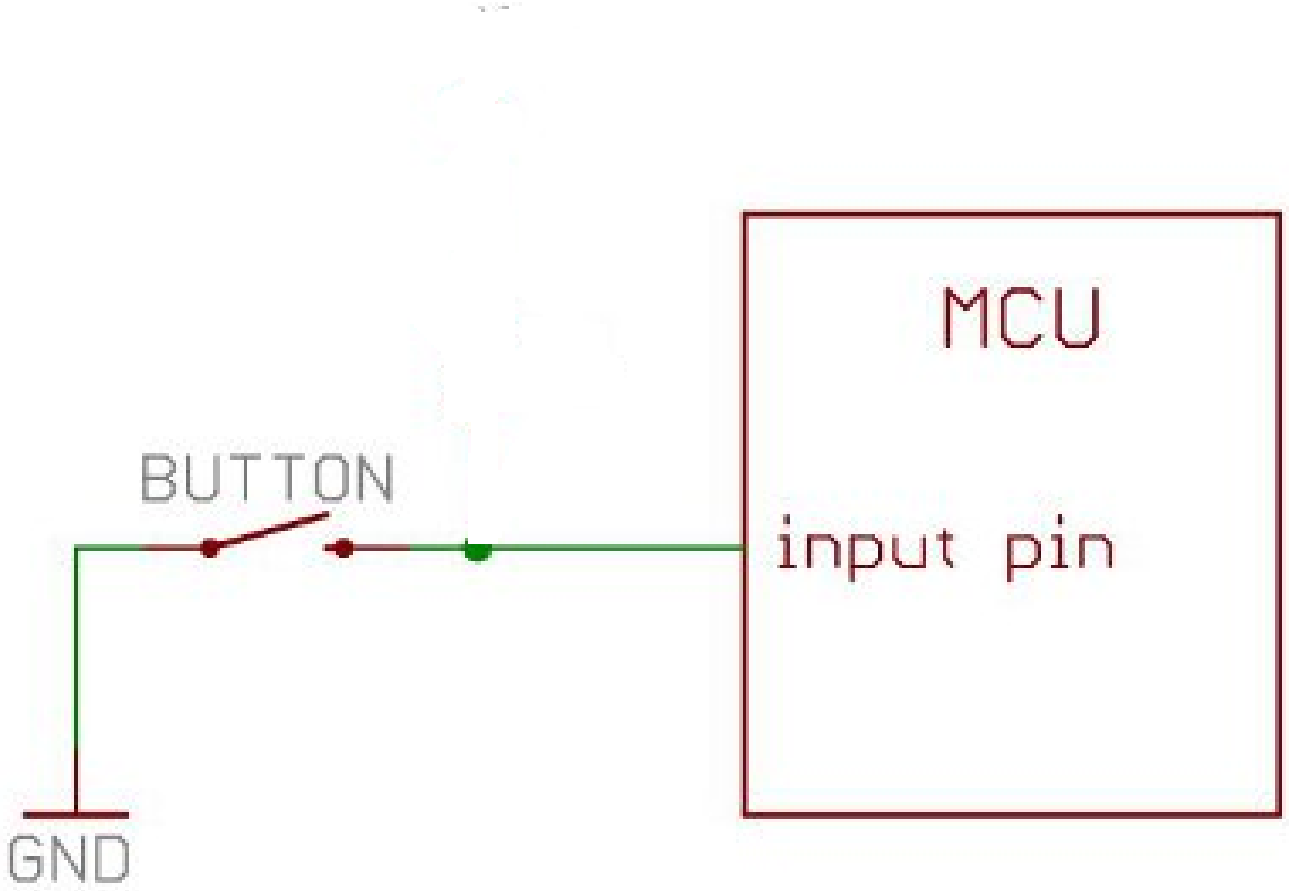
How do you Configure the Green LED?



- Port 2, Pin 1:
  - P2SEL1.1 = ?
  - P2SEL0.1 = ?
  - P2DIR.1 = ?

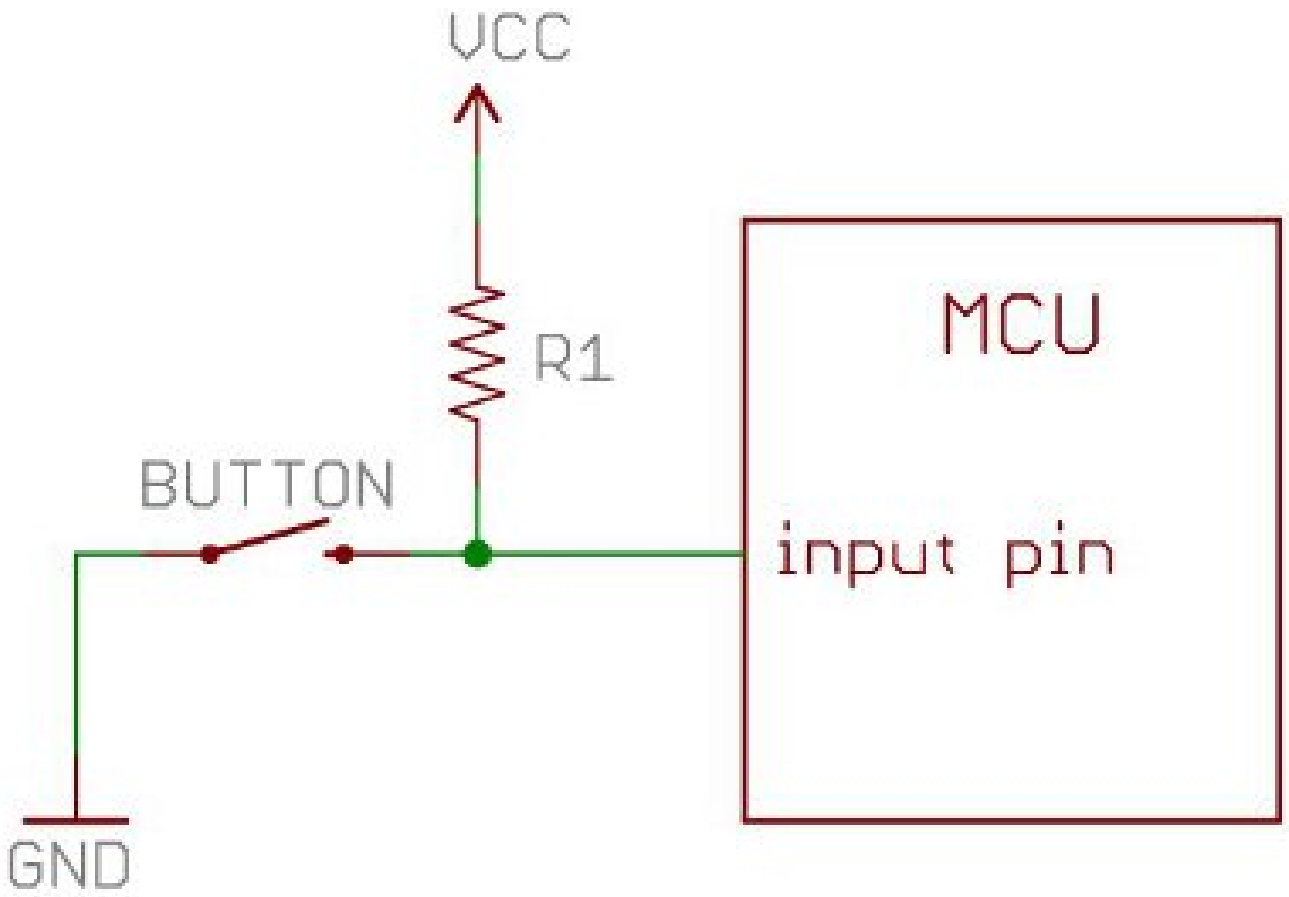
Why do we Need Pullup Resistors?

What is the value of the input? 0 or 1?



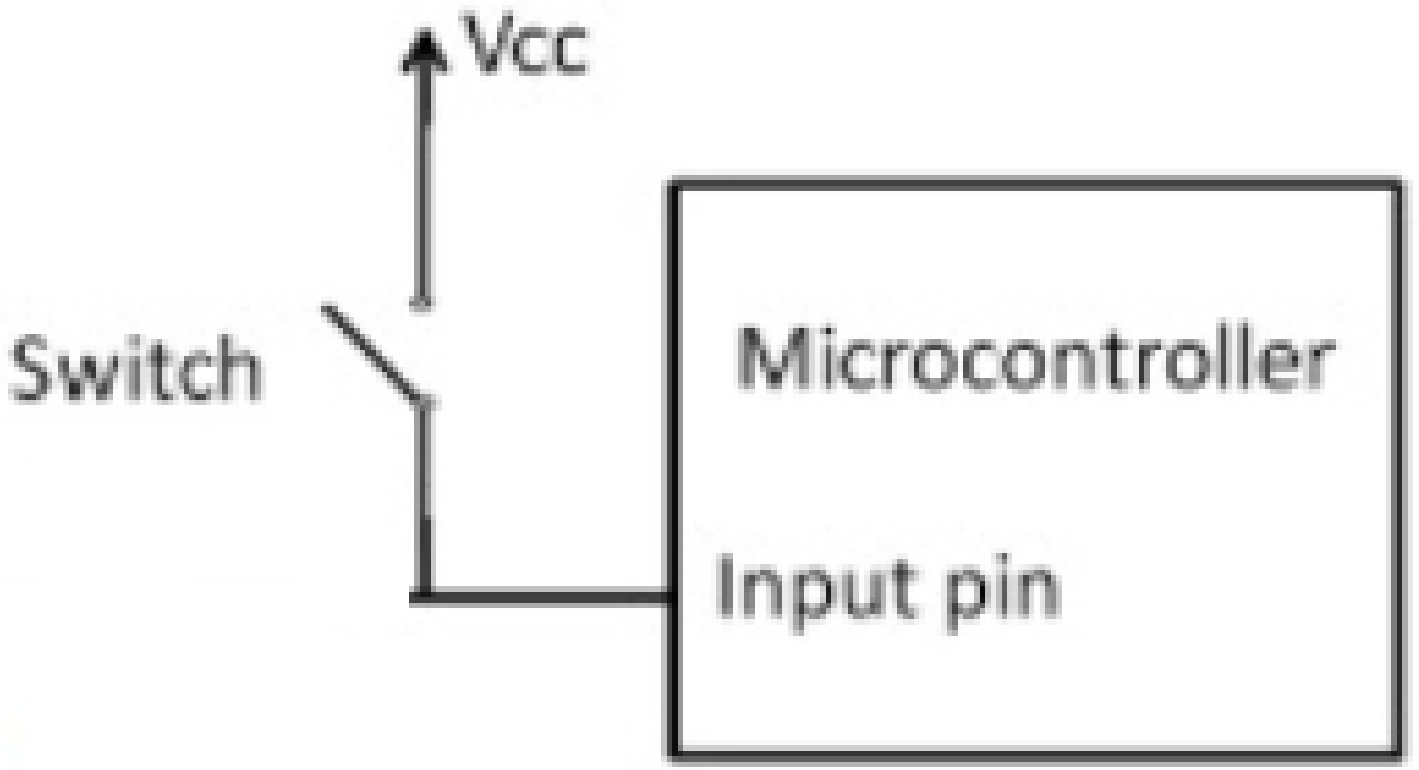
Why do we Need Pullup Resistors?

What is the value of the input? 0 or 1?



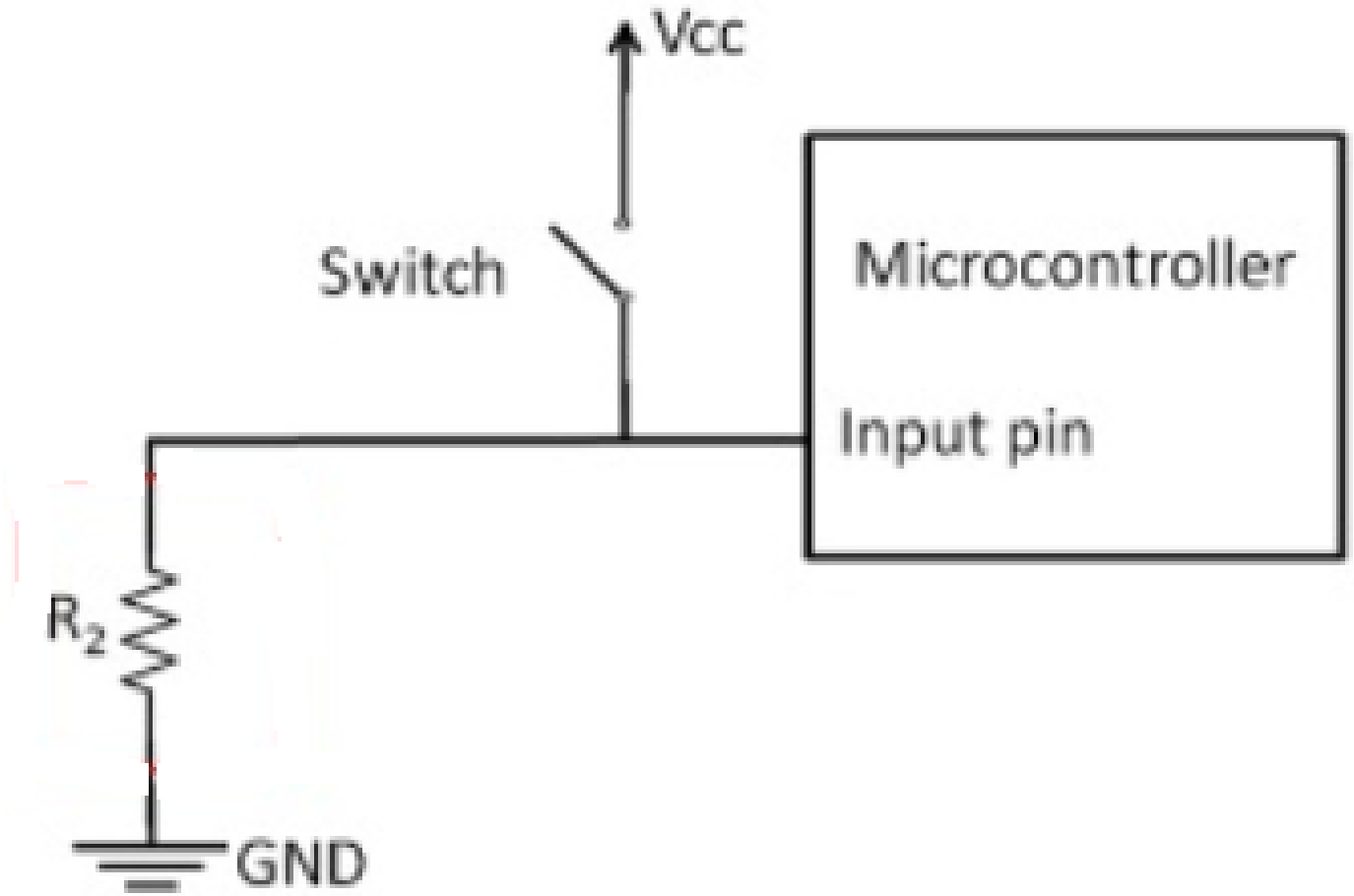
Why do we Need Pulldown Resistors?

What is the value of the input? 0 or 1?



Why do we Need Pulldown Resistors?

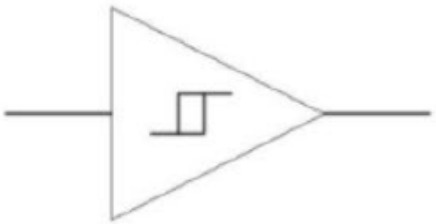
What is the value of the input? 0 or 1?



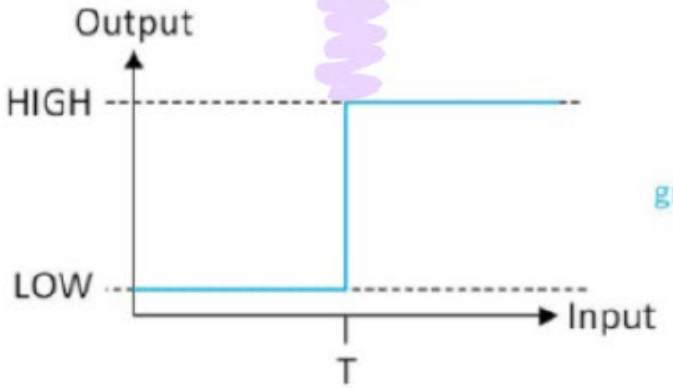
Why do we Need Schmitt Trigger Devices?

A Schmitt trigger converts a noisy digital input to a definitive 0 or 1

Schmitt Trigger
Circuit symbol

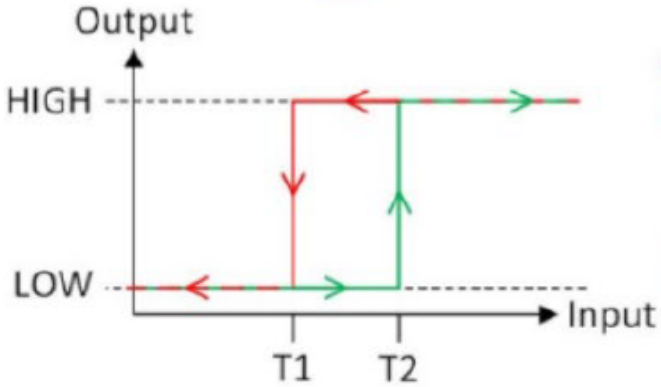


If we used only one threshold...



Output is HIGH when input is greater than threshold T and LOW when input is lower than T

Schmitt Trigger uses two thresholds



When the output is already high, the input should drop below T1 in order to switch the status of the output.

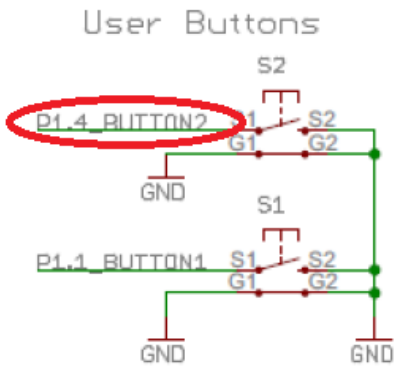
When the output is already low, the input should go above T2 in order to switch the status of the output.

Configuring an Input Button

Input Button

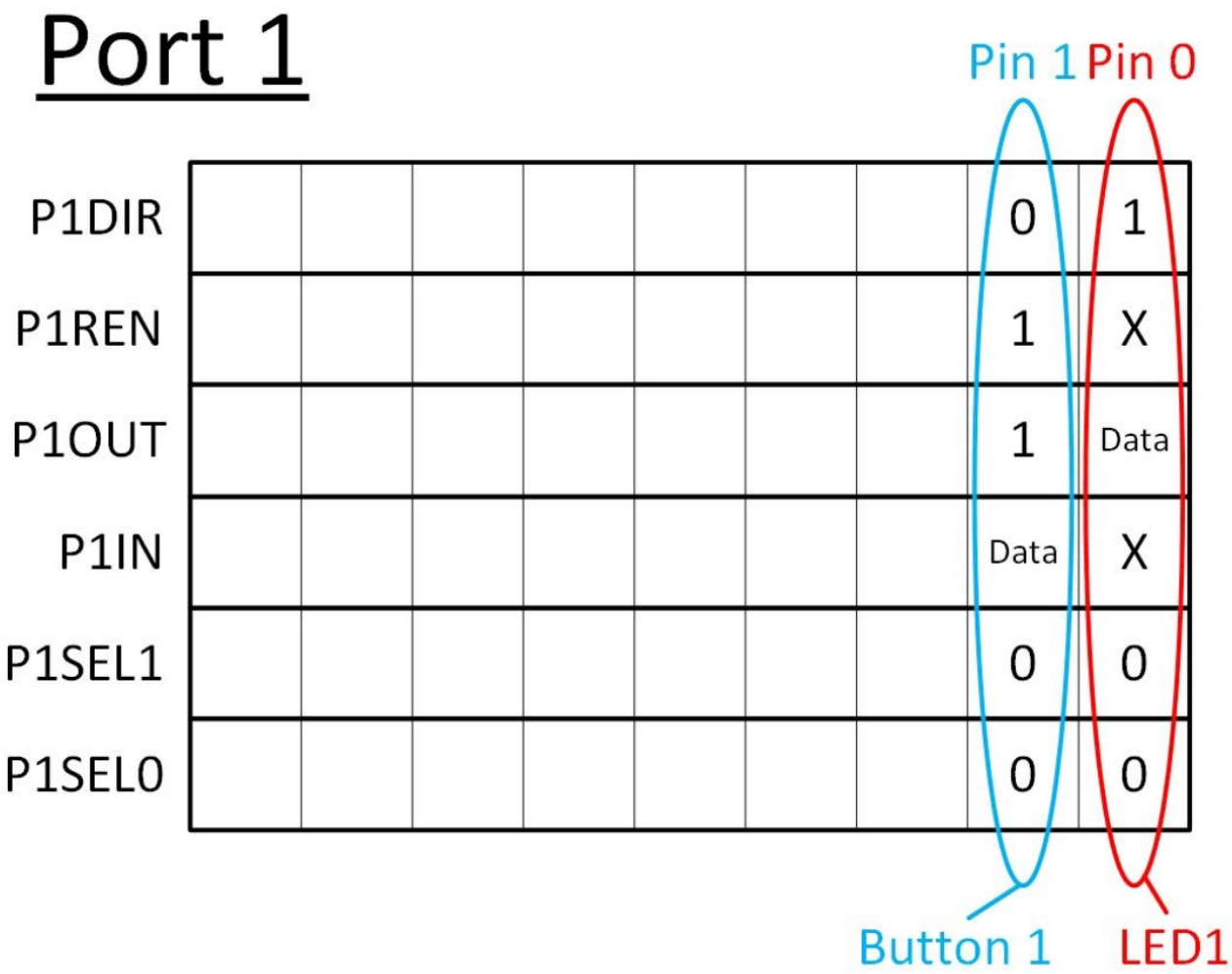
- When pressed down, the voltage is 0 (ground)
- When not pressed, the value is unknown or high-impedance
  - PyREN: Resistor Enable = 1
  - PyOUT = 1 creates a pull-up resistor

How do you Configure the Button #2?



- Port 1, Pin 4:
  - P1SEL1.4 = ?
  - P1SEL0.4 = ?
  - P1DIR.4 = ?
  - P1REN.4 = ?
  - P1OUT.4 = ?

Port 1 has Input and Output Pins



Let's Put the GPIO to USE

- Read the state of pushbotton switch S1
- Turn on LED1 whenever S1 is pressed

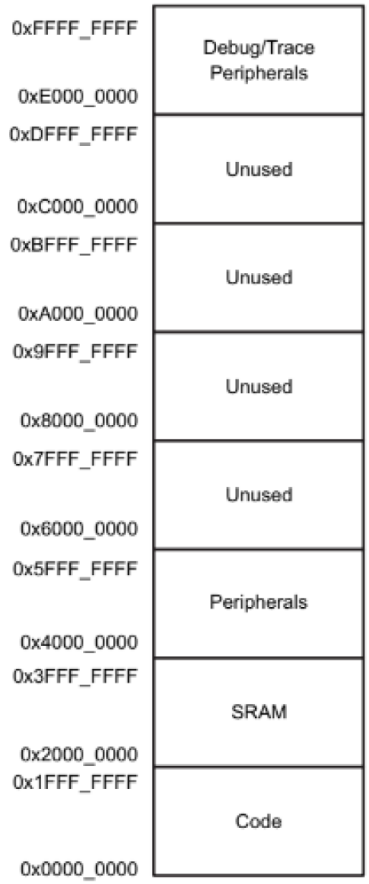
Psuedo Code

- Initialize the GPIO bits associated with S1 and LED1
- In an infinite loop:
  - Read the state of pushbutton switch S1
  - If S1 is pressed
    - Turn ON LED1

    Else
    - Turn OFF LED1

MSP432 Memory Map

- Every address is 32 bits
- Here are the different "zones" of memory
- The CPU communicates with each peripheral through memory-mapped registers



Device Memory Zones

Addresses for GPIO Ports

Port Registers Relative to 0x4000_4C00

REGISTER NAME	ACRONYM	OFFSET
Port 1 Input	P1IN	000h
Port 2 Input	P2IN	001h
Port 1 Output	P1OUT	002h
Port 2 Output	P2OUT	003h
Port 1 Direction	P1DIR	004h
Port 2 Direction	P2DIR	005h
Port 1 Resistor Enable	P1REN	006h
Port 2 Resistor Enable	P2REN	007h
Port 2 Drive Strength	P2DS	009h
Port 1 Select 0	P1SEL0	00Ah
Port 2 Select 0	P2SEL0	00Bh
Port 1 Select 1	P1SEL1	00Ch
Port 2 Select 1	P2SEL1	00Dh
Port 1 Interrupt Vector	P1IV	00Eh
Port 1 Complement Selection	P1SELC	016h
Port 2 Complement Selection	P2SELC	017h
Port 1 Interrupt Edge Select	P1IES	018h
Port 2 Interrupt Edge Select	P2IES	019h
Port 1 Interrupt Enable	P1IE	01Ah
Port 2 Interrupt Enable	P2IE	01Bh
Port 1 Interrupt Flag	P1IFG	01Ch
Port 2 Interrupt Flag	P2IFG	01Dh
Port 2 Interrupt Vector	P2IV	01Eh
Port 3 Input	P3IN	020h

Let's Focus on LED1: P1.0

- What is the address of GPIO register P1SEL0?

Port 2 Drive Strength	P2DS	009h
Port 1 Select 0	P1SEL0	00Ah
Port 2 Select 0	P2SEL0	00Bh

  The address of GPIO register P1SEL0 = 0x4000_4C00 + 00Ah = 0x4000_4C0A

- Write C code to clear the corresponding bit of P1SEL0:

```
#define LED_MASK (1) // Why 1?  1 = 2^0 (Pin 0)
...
unsigned char *P1SEL0_addr =  (unsigned char *) 0x40004c0a;
*P1SEL0_addr &= ~LED_MASK;
```

Why `char` instead of `int`?

Write the Complete Program

```
#define LED1_MASK 1
#define S1_MASK 2
int main(void){
    // set up the register addresses for Port 1
    unsigned char *P1SEL0_addr =  ... ;
    unsigned char *P1SEL1_addr =  ... ;
    unsigned char *P1DIR_addr =  ...
    unsigned char *P1REN_addr =  ... ;
    unsigned char *P1IN_addr =  ... ;
    unsigned char *P1OUT_addr =  ... ;
    // configure LED1 bit for digital output (P1.0)
    *P1SEL0_addr &=  ... ;
    *P1SEL1_addr &= ... ;
    *P1DIR_addr |= ... ;
    // configure switch S1 for digital input (P1.1)
    *P1SEL0_addr &= ... ;
    *P1SEL1_addr &= ... ;
    *P1DIR_addr &= ... ;
    *P1REN_addr |= ... ;
    *P1OUT_addr |= ... ;
    // loop forever
    while(1){
        if (... S1 Pressed ...)
            ... Turn on LED1 ...;
        else
            ... Turn off LED1;
    }
}
```

Observation

- This example was tedious to write because it was very low-level, close to the hardware code
- In the next module we will learn to use the TI Driver Library that provides a higher level of abstraction giving us more distance from the hardware

Using the TI Driver Libary to Control the GPIO

```
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

int main(void)
{
    . . .

    // Set P1.0 to output direction
    GPIO_setAsOutputPin( GPIO_PORT_P1, GPIO_PIN0 );

    . . .

    while(1)
    {
        // Toggle P1.0 output
        GPIO_toggleOutputOnPin( GPIO_PORT_P1, GPIO_PIN0 );

        . . .
    }
}
```

Useful Constants

```
/*************************************************************************************
* Definition of standard bits                                                        *
*************************************************************************************/
#define BIT0                                        (uint16_t)(0x0001)
#define BIT1                                        (uint16_t)(0x0002)
#define BIT2                                        (uint16_t)(0x0004)
#define BIT3                                        (uint16_t)(0x0008)
#define BIT4                                        (uint16_t)(0x0010)
#define BIT5                                        (uint16_t)(0x0020)
#define BIT6                                        (uint16_t)(0x0040)
#define BIT7                                        (uint16_t)(0x0080)
#define BIT8                                        (uint16_t)(0x0100)
#define BIT9                                        (uint16_t)(0x0200)
#define BITA                                        (uint16_t)(0x0400)
#define BITB                                        (uint16_t)(0x0800)
#define BITC                                        (uint16_t)(0x1000)
#define BITD                                        (uint16_t)(0x2000)
#define BITE                                        (uint16_t)(0x4000)
#define BITF                                        (uint16_t)(0x8000)
#define BIT(x)                                    ((uint16_t)1 << (x))
```

Defined in `simplelink_msp432p4_sdk_3_40_01_02/source/ti/devices/msp432p4xx/inc/msp432p401r.h`

Summary

- GPIO hardware
- Hardware interfacing issues
  - Pull-up/down resistors, Schmitt triggers, tri-state buffers
- Configuring GPIO pins

PxDIR	PxREN	PxOUT	I/O Configuration
0	0	x	Input
0	1	0	Input with pulldown resistor
0	1	1	Input with pullup resistor
1	x	x	Output

- Using the memory map to write code